

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

ONLINE CODE JUDGE

Database Systems Engineering & Distributed Backend Development (25CS1302E)

ACADEMIC SESSION

Y25 | 2026–2027

Trimester 4 • Section 10 • Team 19

EVALUATION STAGE

Project Review 1

Concept, Architecture & Scope Definition

TEAM DETAILS

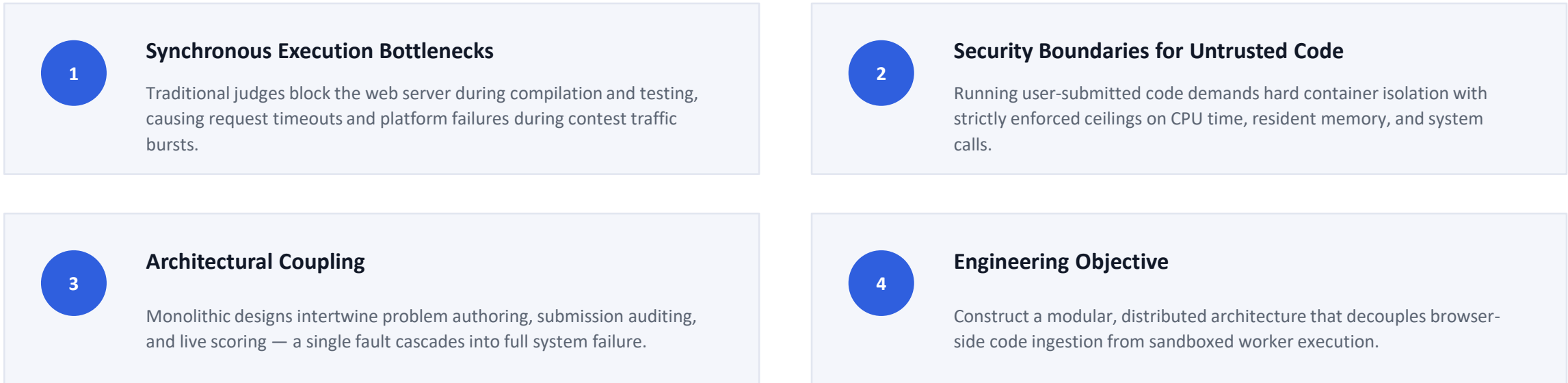
Bhargava Raaj Vamsi (2520090169)

Tarun (2520090020)

Hanish Kumar (2520090107)

Why Existing Judges Break Under Load

System rationale & context



Limitations of Conventional Online Judges

What current platforms get wrong

- Request-response execution: code compiles and runs inline with the HTTP request, so the web server stalls under concurrent load.
- Weak isolation: many legacy judges rely on OS-level sandboxing (chroot, ulimit) instead of container-level isolation — a real security exposure.
- No submission queue: without asynchronous dispatch, contest traffic spikes overwhelm the server and cause dropped submissions.

- Stale leaderboards: ranks are recomputed on a fixed interval rather than updated in real time, so standings lag behind actual scores.
- Little to no plagiarism detection: duplicate or lightly modified submissions usually go unflagged.
- Tight coupling: problem authoring, judging, and scoring live in one codebase, so one failure can take the whole platform down.

A Decoupled, Sandboxed, Real-Time Judge

How the new architecture resolves each gap

Microservice Decoupling

API Gateway, Judge Worker, and Contest Service communicate asynchronously over a Redis-backed queue — no shared failure domain.

Docker-Sandboxed Execution

Each submission runs in an isolated container with enforced CPU, memory, and time ceilings, protecting the host from untrusted code.

Asynchronous Job Queue

Redis + BullMQ dispatch submissions off the request path, keeping the platform responsive during contest traffic bursts.

Real-Time Leaderboards

Redis sorted sets drive $O(\log n)$ rank updates, so standings reflect verdicts the instant they're computed.

AST-Based Plagiarism Detection

Token fingerprinting and AST diffing flag suspiciously similar submissions within a contest window.

Multi-Language Support

A browser-based Monaco Editor accepts C++, Python, Java, and JavaScript, judged against structured, multi-case test suites.

What This Trimester's Build Sets Out to Achieve

1

Design a non-blocking submission pipeline using a Redis-backed BullMQ job queue.

2

Implement Docker container sandboxing with enforced CPU, memory, and syscall constraints per submission.

3

Architect decoupled microservices — API Gateway, Judge Worker, and Contest Management — over asynchronous messaging.

4

Build a real-time leaderboard using Redis sorted sets for $O(\log n)$ rank updates at contest scale.

5

Develop AST-based plagiarism detection through token fingerprinting across a contest window.

6

Support multi-language execution (C++, Python, Java, JavaScript) via a browser-based Monaco Editor.

Project Boundaries for This Build

IN SCOPE

- Browser-based Monaco code editor for submissions
- Problem authoring with structured, multi-case test suites
- Multi-language judging: C++, Python, Java, JavaScript
- Structured verdicts — AC, WA, TLE, MLE, RE, CE
- Contest session management with time boundaries
- Real-time leaderboard via Redis sorted sets
- AST-based plagiarism detection

OUT OF SCOPE (THIS PHASE)

- Native mobile applications
- Payment or subscription billing features
- AI-generated code review or hints
- Video-based proctoring
- Multi-tenant SaaS onboarding for external orgs

Frameworks, Platforms & Tools



Frontend — React.js

React.js UI with the Monaco Editor — the engine behind VS Code — for in-browser coding.



Backend Runtime — Node.js

Node.js + Express.js structured as microservices: API Gateway, Judge Worker, Contest Service.



Persistent Store — MongoDB

MongoDB holds users, problems, submissions, and contest data with leaderboard-optimized schemas.



Queue & Cache — Redis

Redis powers the BullMQ submission queue and in-memory sorted-set leaderboards.



Sandboxing — Docker

Docker spins up an isolated container per submission with strict CPU, memory, and time limits.



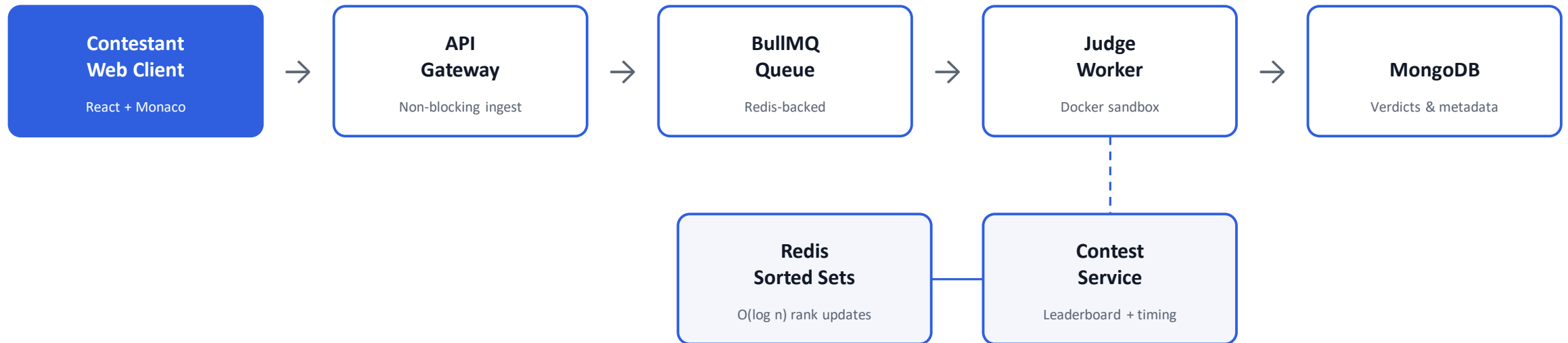
Plagiarism Engine

AST-based diffing and token fingerprinting detect suspiciously similar submissions.

Deployment: Docker Compose orchestrates all services for consistent local and production environments.

System Flow: Client to Verdict

Modular microservices connected over asynchronous messaging



Submissions enter through the API Gateway and are queued asynchronously via BullMQ, so the gateway never blocks on compilation. Judge Workers execute code inside isolated Docker containers, persist structured verdicts to MongoDB, and notify the Contest Service, which maintains real-time rankings in Redis sorted sets.

STATUS

Abstract Submitted

Project Abstract Submission Form — Database Systems Engineering & Distributed Backend Development (25CS1302E)

Team No.	19
Section	10
Academic Session	Y25 2026–2027, Trimester 4
Evaluation Stage	Project Review 1
Team	Bhargava Raaj Vamsi (2520090169) • Tarun (2520090020) • Hanish Kumar (2520090107)

Next: Project Review 2 — Frontend & 3NF Data Design